M1 Robot software foundations

M1 • 11 Oct 2026 - 03 Nov 2026 • 75 hours
Python, C++ and Rust: a body, one joint, observation and fault investigation

Deliverable: telemetry-core v1.0. One repository with a shared IMU and joint data structure, three command-line utilities (CLIs), a Python analyser and a main C++ processor. A generator produces data; the test bench displays roll and joint-angle error, detects invalid rows and interrupted transmission, and saves a log for repeating the experiment. This prepares you for legs and fingers; actuators and physics simulation are not yet needed.

Block	Dates in 2026	Analysis	Practice	Deliverable
1	11-16 October	10 h	10 h	Environment, snapshot and Python
2	18-23 October	10 h	10 h	C++ and logger
3	25-30 October	10 h	10 h	Faults, tests and Rust
4	1-3 November	5 h	10 h	Streaming, timeout and replay
Total	M1	35 h	40 h	75 hours

Calendar. Four Sundays of 10 h and 17 weekdays of 2 h give 74 h. One of the programme's 12 additional hours is assigned to **Monday 2 November: 3 h**; the source does not date this hour. Saturdays are free. In the source, M2 also starts on 3 November, with no boundary specified by hour; its calendar is not recalculated. Detailed dates and tasks are on pp. 11-12.

Work rhythm and scope

Weekdays: 120 min - restore context 10, read 45, work through an example 35, explain 20, record results 10. Sunday: five 2 h blocks - readiness criterion 10, implementation 90, verification 20 min. Breaks of 15-30 min and lunch are outside the 75 h. Reading, glossary work and setup are included.

Keep one task on a board labelled 'Next / In progress / Verified'. Log the date, minutes, scenario, random-number generator seed, axes, units, commit, parameters and command. After 30 min without progress, simplify the example; after 60 min, describe the problem and change sources. Update README at each block's end. The main processor is C++; cut extensions before core tests.

Acceptance checks: evidence to save

Provide an axes and joint-zero diagram, rest and 15° roll recordings, q_ref and q plots, and a NO_RESPONSE event without guessing its cause. Check all three CLIs and count gaps, rejected and dropped separately. The watchdog must trigger during silence; duplicates must not refresh its timer, and FAULT must not clear itself. Include raw-recording replay; GDB, ASan/UBSan, and TSan or its limitations; a clean Ubuntu build; timing measurements, CLI medians, p50/p95/p99; queue sizes and deliberately slow logging.

Save six scenarios, expected results calculated by hand, plots, events, a report and commands. Total your hours and identify three skills, two mistakes and unfinished tasks; repeat the experiment independently after AI assistance. Multiple joints will require names and aligned timing. Balance, kinematics, state estimation, physical constraints and torque control come later.

Motion fundamentals are compulsory from M1: link, joint, degree of freedom, configuration and state; the distinction between kinematics, dynamics and control. Explanations are on p. 13 and the first mechanism calculation on p. 15. It belongs to the 'axes, units, RobotState' block on 11 October.

A body and one-joint test bench

A shared state snapshot: from reading fields to observing a robot

Mechanism and coordinates

The model has a body with an IMU and one revolute joint, such as a leg's knee or a finger joint. Use the IMU accelerometer readings, namely specific force. Its axes align with the body axes: x forwards, y left, z upwards, forming a right-handed frame. Measure specific force in m/s², joint angle in radians and current in amperes. Draw and label the joint zero and positive direction in README.

A snapshot contains one generator step's data. All fields share the timestamp t_us; this is a test-bench simplification. Real IMUs and encoders may have different rates and clocks. Here, current i_a is merely a recorded signal; it is not actuator torque. q_ref_rad is the requested angle and q_rad the encoder measurement.

```
seq,t_us,ax,ay,az,q_ref_rad,q_rad,i_a
0,0,0.0,0.0,9.81,0.0,0.0,0.2
1,10000,0.0,0.0,9.81,0.01,0.008,0.2
```

Data and error contract

Use unquoted UTF-8 CSV with the exact header and 8 fields. seq and t_us are ASCII integers in 0…2^63-1. Other fields are finite decimal numbers; signs, decimal points and exponents are allowed. Reject hex, underscores and NaN/Inf. Trim surrounding whitespace and require complete parsing. Both angles of the teaching joint are limited to [-1; 1] rad.

For accepted snapshots, seq and t_us increase strictly. An invalid row does not update the accepted state. A seq gap is a diagnostic event, not a reason to reject the next snapshot. Resetting the counter requires a new recording; M1 does not require automatic reset detection.

Check	Result
Header, file	Stop with code 2 if the header is wrong or the file cannot be read.
Row or fields	Reject as blank, field_count, number, nonfinite, joint_range, sequence or timestamp, in that priority order.
CLI result	Code 0: all rows valid; 1: some rejected. JSON to stdout, reasons to stderr.
Summary	rows_total, accepted, rejected, errors_by_reason; first/last_t_us; mean_ax/ay/az; max_abs_joint_error. Empty metrics: null.

Experiment set

Create separate files of 1 000 snapshots at 10 000 µs intervals: **rest**; **static 15° roll**; **smooth joint motion**; **no response after seq=200**; **seq=40…49 gap**; **bad rows**. Use a separate streaming test with a pause to test real clock progression. Verification files contain no noise; noise with seed=42 is an additional experiment.

All language implementations read the same files. Calculate expected results for small experiments by hand. Integers and error reasons must match exactly; metrics use an absolute tolerance of 10^-9 for these data. Shared layout: python/, cpp/, rust/, scenarios/, fixtures/, tests/, reports/.

Linux and reproducible experiments

Learn to run and investigate a robotics experiment

Topics to study

• **Files and the shell.** Paths, working directory, permissions, environment variables, quoting, stdin/stdout/stderr, redirection, pipes and exit codes. Commands: pwd, ls, cd, mkdir, cp, mv, less, head, tail, rg, chmod, env.

• **Processes.** PID, foreground and background execution, ps, top, SIGINT/SIGTERM. Learn how to stop the generator while allowing the logger to close its file, and why kill -9 prevents normal cleanup.

• **Python environment.** Interpreter, virtual environment, pyproject.toml, uv.lock, uv run and dependency restoration. The project needs numpy, matplotlib and pytest.

• **Git.** Index, commit, diff, branch and merge; Pro Git 2.1-2.5, 3.1-3.2. Preserve the history of scenario parameters and changes to sensor-data processing.

Workstation setup

On an Apple Silicon Mac, use the planned Ubuntu 24.04 ARM64 in UTM for primary runs. If Ubuntu is already available, use it. You need a terminal, Git, uv/Python, a C++ compiler, CMake, GDB, clang-format and Rustup/Cargo. These experiments require neither ROS nor a physics simulator.

Record versions, architecture and commands in README. Add .venv, build, target, caches and large results to .gitignore. Keep small verification recordings in Git; generate large ones. For every experiment, save the scenario, axis and unit configuration, seed, commit and result.

Practical tasks

L1 Run an experiment. Generate a stationary-robot recording, analyse it, and save the JSON summary and errors separately. Repeat from another directory using explicit paths. Then supply a missing file: this must produce a clear error rather than an empty successful report.

L2 Process lifecycle. Make the generator emit one row every 10 ms with an explicit flush; actual intervals may vary. Pipe it into a simple Python logger. Stop the generator with SIGINT, wait for EOF and check the last saved row. The final multithreaded logger comes later.

L3 Configuration history. Create a branch to correct the y-axis sign in the tilt scenario. Change the data and expected result together, explaining why in the commit. Create a threshold-value conflict in a separate practice configuration file, resolve it and rerun the relevant test.

L4 Replay. In a project copy, restore dependencies from the lockfile and repeat the 'no response' experiment. The command, data version and result must match. A VM snapshot helps restore the environment, but does not replace project history and an experiment description.

Connection to later robotics work

These skills matter when a sensor or driver runs as a separate process, an experiment runs on an onboard computer, and a fault must be reproduced on a laptop. Save a reproducible experiment folder and explain what happens when each process stops.

Python and motion diagnostics

Five tasks using body and joint data

Study before practice

Python: functions, dataclass, dict/list, generators, with, modules, exceptions, argparse, pathlib and JSON. NumPy: arrays, shape/dtype, columns, masks, abs, mean, max, isfinite, sin/cos and arctan2. Matplotlib: Figure/Axes, multiple lines, labels, legends and savefig. Apply each construct to a RobotState snapshot.

P1 Generate data for known motion

For rest, set f=(ax,ay,az)=(0,0,9.81), q_ref=q=0, i=0.2. For static roll φ=15°: f=(0,g·sinφ,g·cosφ), g=9.81; convert φ to radians for sin/cos. The joint is stationary in both experiments. These are ideal data with known answers.

For joint motion, set q_ref=min(0.8, 0.002·seq), q=max(0, q_ref-0.02), i=0.2. In the 'no response' variant, after seq=200 hold q at its value at 200 while q_ref keeps changing. Record this boundary in the configuration. This is a predefined motion trace, not a physical motor model.

P2 Read and validate a snapshot

Implement parse_row and sequence validation separately. Stream the file using with, recording rejection reasons and maintaining accepted+rejected=rows_total. Distinguish an unrepresentable number, a packet gap and mechanically suspicious behaviour: these are different events. Update the last valid state only after all checks.

P3 Plot body data

Plot ax, ay, az and the specific-force magnitude against time. In static experiments, estimate roll as φ_est=atan2(ay,az): expect 0° and 15°, with a 0.1° tolerance without noise. Add an artificial 2 m/s² to ay for the stationary body and demonstrate the false tilt. Conclude that linear acceleration and vibration interfere with accelerometer-only tilt estimation; yaw cannot be determined this way.

P4 Detect a joint that does not respond

Plot q_ref and q, then e=q_ref-q. Find max|e| and mean |e|. Flag **NO_RESPONSE** if, in a 10-snapshot window with consecutive seq values, every |e|>0.1 rad and max(q)-min(q)≤0.005 rad. Restart the window after a gap. These are teaching thresholds; 10 samples specify a sample set, not an exact duration.

Compare the event with the scenario label. q alone cannot distinguish a frozen encoder, a jammed mechanism and a disconnected motor: report 'no expected response'. Plot current alongside as an additional observation, but do not treat it as proof of the cause.

P5 Find transmission gaps

Remove seq=40…49 from a clean file: 990 of 1 000 snapshots remain, with 10 seq numbers missing. Next corrupt one row without removing it: rejected=1. A number missing among accepted snapshots does not prove transmission packet loss: validation may have rejected a row. Account for these cases separately.

Deliverables: a JSON summary, body and joint plots, events.json with seq and reason, and expected results calculated by hand for every experiment. Python analyses events; C++ and Rust implement the shared contract, not the entire analysis.

Modern C++ and resource management

Object lifetime, resource ownership and porting the data parser

Study in this order

• **Building a programme.** Source, header, translation unit, object file and linking. Examine one compilation error and one linking error in the sensor processor. Study declarations and definitions, include guards or pragma once.

• **Values and references.** Initialisation, automatic lifetime, scope, const, passing by value and by const T&. Compare pointers and references. Understand why a reference to a local variable becomes invalid.

• **RAII.** The constructor establishes a valid state; the destructor releases the resource. A file closes both on function exit and on an exception. Usually express ownership with an object, std::vector or std::unique_ptr; use shared_ptr only when shared ownership is actually needed.

• **Standard library.** std::string, std::vector, std::array, iterators, range-for, algorithms and std::optional. Determine when modifying a vector invalidates references. Study string_view as a non-owning view with a limited lifetime.

• **Copying and moving.** Understand copy/move behaviour before studying std::move. It permits move selection but does not itself transfer bytes. Do not build a complex class hierarchy for a RobotState structure containing a few numbers.

Practical tasks with robot data

C1 State snapshot. Create RobotState with the contract's eight fields. Put the parser, structure and CLI in separate files. Pass a snapshot by value and by const reference; explain who owns it and why a consumer must not retain a reference to a reused sensor buffer.

C2 Data reception. Port snapshot validation from Python, including the angle range and seq ordering. Run rest, motion and corrupted recordings. Accepted rows, counters, mean ax/ay/az and max|q_ref-q| must match; NO_RESPONSE analysis remains in Python.

C3 Logger resource. Wrap the output file in a RAII object. Save 10 snapshots; exit via return, then via an exception after the fifth. Verify that the object is destroyed and the error remains visible. Check stream state on explicit close: object destruction alone does not prove a successful disk write.

C4 Sensor buffer. In a separate experiment, retain a string_view of an IMU row, then reuse the original buffer. Show the changed contents; separately reproduce an invalid reference after vector reallocation. Fix the transfer by parsing the row into a RobotState that owns its data before reusing the buffer. Save ASan diagnostics where the error is detected.

Readiness for the second checkpoint deliverable

C++ produces the counters and means required by the contract. Explain each object's owner and why the main code has no dangling references. C++17 is sufficient for M1; record the standard and features used in CMake. Template metaprogramming, custom allocators and complex design patterns are unnecessary.

CMake, debugging and diagnostics

Reproduce an error, find its cause and verify the fix

Topics to study

In the official CMake Tutorial, study building an executable and a library, separating subdirectories, target commands, and Testing and CTest. Understand the relationship between add_library, add_executable, target_link_libraries, target_include_directories and target_compile_features. Distinguish PRIVATE, PUBLIC and INTERFACE using one dependency example; do not study the entire CMake language.

The project needs a telemetry library, a CLI and a test build target. Enable -Wall -Wextra -Wpedantic for GCC/Clang. Use Debug for debugging and Release for speed comparisons. clang-format establishes a consistent style; formatting does not establish programme correctness.

```
cmake -S cpp -B build/debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/debug
ctest --test-dir build/debug --output-on-failure
cmake -S cpp -B build/release -DCMAKE_BUILD_TYPE=Release
cmake --build build/release
```

These commands assume a configured CMakeLists.txt. Describe GoogleTest integration and test-data paths in README; pin the dependency version for reproducible builds.

Debugging a concrete scenario

D1 Incorrect units. In a practice analyser version, pass 15 as radians instead of 15°. In GDB, set a breakpoint at the q_ref check, inspect the value and stack, and practise run, next, step, print and backtrace. The range check must reject this command; fix the conversion and add a test. Small unit errors may remain within range, so this check alone is insufficient.

D2 Corrupted snapshot. In a separate faulty C++ version, access q after receiving a row with too few fields. Examine the crash stack and diagnostic-tool message, then check the length before accessing fields. Save the input file and test. Study core dumps using this example; if dump saving is disabled, debug directly in GDB.

Separate diagnostic builds

Check	Practice experiment	Save
ASan and UBSan	Reused sensor buffer, invalid field index and integer timestamp overflow.	Report excerpt, cause and corrected version.
TSan	Reader and monitor update shared RobotState without a mutex, then fix it.	Command, environment and race-check result.
Regular tests	Repeat the checks on the corrected Debug build.	Passing report and a test for the original error.

ASan and UBSan are usually compatible; build TSan separately. Both compiler and linker need the flags: -g and, where necessary, -fno-omit-frame-pointer. Do not measure speed with diagnostics enabled. If the environment does not support TSan, record that limitation and retain queue tests; race checking remains incomplete.

Robot fault tests

Verify the CSV format and the evidence behind conclusions about the robot

Testing-tool topics

pytest: assertions, fixtures, tmp_path, raises and parametrize. GoogleTest: TEST, ASSERT/EXPECT, TEST_F and execution through CTest. In Rust, use unit tests and cargo test. A Python programme that launches all three CLIs checks the shared files and expected.json; separate Python tests check robot-behaviour diagnostics.

Scenario	Expectation	What it checks
S1 Rest and roll	For error-free static data, roll 0°/15° within 0.1°; magnitude 9.81.	Axes, units and method limitations.
S2 Motion	Smooth recording within range; max|e| ≤ 0.02 rad with numerical tolerance; no NO_RESPONSE.	Distinction between command and measurement.
S3 No response	After q is held at seq=200, NO_RESPONSE occurs under rule P4.	Fault indication without an unsupported conclusion about its cause.
S4 Missing records	Remove seq=40…49: accepted=990, rejected=0, accepted-sequence gap 10.	Missing snapshots counted separately from bad rows.
S5 Invalid input	Blank row, 7/9 fields, NaN/Inf, 1_000, hex, angle 1.01 rad, repeated seq/t_us.	Rejection reason; accepted state remains unchanged.
S6 Stream disappearance	More than 100 ms since a new snapshot; FAULT and simulated sending disabled.	Periodic checking without new messages.

T1 Calculate expected answers by hand

Save the generator configuration and a small example with its expected answer for each scenario. Separately convert 0°, 90° and 180° to 0, π/2 and π; the working joint remains limited to ±1 rad. The 90°/180° cases test the conversion function and must not become joint commands.

Test ±1 rad, the 0.1 rad threshold, an empty file and CRLF. NO_RESPONSE requires cases with 9 and 10 threshold exceedances, followed by a seq gap. Calculate expectations independently of the code under test.

T2 Introduce errors deliberately

Remove the degrees-to-radians conversion, refresh the watchdog on an invalid packet, and allow NaN in q. A test must detect each defect. Correct the code and explain how the error affects the represented position of a leg or finger.

T3 Controlled time and streaming

Supply now manually in the watchdog test: last snapshot at 0 ms; tick(100) remains valid; tick(101) produces FAULT. A duplicate at 90 ms does not refresh the valid-snapshot receipt time. Do not wait real seconds in unit tests. Separately test the queue for EOF, a full buffer, write failure and termination of both threads.

Scope: about 20-25 compact cases. Test the parser in each language; CLI equivalence through a shared programme; sensor events in Python; and the queue and watchdog in C++. Three complete system implementations are not required.

Rust as a practical tool

Understand ownership and errors by validating familiar robot recordings

Required Rust Book chapters

Chapter 3: selected language constructs; chapter 4: ownership, references and borrowing; chapter 5: structs; chapter 6: enum and match; chapter 8: vector, string and hash map as needed; chapter 9: Result and error handling; chapter 11: testing. From chapters 12-13, use only CLI arguments, file reading and iterators. A conceptual overview of chapter 16 on threads is sufficient in M1.

Identify who owns a String, where a value moves, how long a reference lives, why mutable borrowing restricts other access, and when Result or Option is needed. Begin with owning structures and simple borrows; defer complex lifetime annotations.

Minimum implementation scope

• **R1.** Create a Cargo project, a RobotState structure and a ParseError enum. Write parse_row(&str) -> Result<RobotState, ParseError>. Check finite values and joint range, and return a clear reason. A corrupted snapshot must not become robot state; unwrap() is unsuitable for handling input failures.

• **R2.** Pass an IMU snapshot row by value, then try logging the same String and read the compiler error. Change the parameter to &str. Separately create conflicting mutable and immutable borrows, then fix their usage scope.

• **R3.** Read a leg or finger log through BufRead and produce the shared JSON summary and exit codes. You may use serde/serde_json for JSON with a pinned Cargo.lock. Do not spend time escaping JSON manually.

• **R4.** Use cargo test to check a bad IMU row, an invalid angle and a normal snapshot; connect the shared verification programme. Run cargo fmt --check and cargo clippy, reading every warning. Use cargo build --release for measurements.

A substantive comparison of Rust and C++

Question	C++	Rust
When is a resource released?	Usually when a RAII object's lifetime ends.	Usually when the owner's lifetime ends, through Drop.
How is an invalid row represented?	An explicit result structure or the chosen exception policy.	Result with an error variant and match or the ? operator.
What happens to unsafe access?	Some errors are detected by analysis, tests and runtime diagnostic tools.	The safe Rust compiler rejects many borrowing errors.

Save two Rust diagnostic examples and the C++ RAII experiment. Explain that ownership checking constrains how code can be written, but does not eliminate logic errors, deadlocks or the need for tests. Compare performance using your own measurements.

Readiness criterion. The Rust utility passes the shared checks, and you can explain ownership errors in your own words. Implement the main processor once, in C++.

Data streaming and update monitoring

Distinguish slow disk writes from an interrupted robot stream

Programme model and study topics

Study threads, shared memory, mutexes, RAII locks and condition_variable with a predicate. A reader thread receives snapshots; a writer thread logs them through a queue of K=64. latest_state stores the accepted snapshot and host_received_at. Read and update all shared state under one mutex; perform file I/O outside the lock.

On normal EOF, close the queue: set closed, wake threads, write the remaining entries and join. A writer error wakes a blocked reader and ends the experiment with an error. The monitor runs in the main thread. Disable the watchdog for file processing; in streaming mode, normal EOF disables commands and stops the monitor before draining the queue.

Q1 Logger overload

In **blocking** mode, the reader waits for space; a slow disk also slows reception of new snapshots. In **drop newest** mode, a new record cannot enter a full queue, but the reader continues updating latest_state. The log can therefore lose rows while the monitor still sees new snapshots. Report missing log entries and reception delays separately, explaining their causes.

For verification file processing, use blocking mode without losses. For streaming, compare K=4 and K=64 with a slowed writer. Check saved seq order and capacity. After normal completion, accepted=enqueued+dropped and enqueued=processed. Count invalid rows separately; after an error, also report unprocessed snapshots.

Q2 Monitor updates independently of packet reception

Roughly every 10 ms, the main thread calls tick(now) and checks now-last_new_valid_receive. After **more than 100 ms**, it logs FAULT and disables simulated command sending. Before the first snapshot, state is INIT and commands are disabled; timing starts when reception begins. A first valid snapshot before timeout changes INIT to OK. Duplicates, old seq values and invalid rows do not refresh the time. The 10/100 ms limits are teaching settings, not real-time guarantees.

In the streaming test, stop updates while the reader keeps waiting. The watchdog must trigger without another message. A new valid snapshot after failure does not automatically clear FAULT: allow reset separately, and only with a current accepted snapshot. No commands are sent to actual actuators; test the command_allowed variable and event log.

Q3 Required checks

• Update RobotState as a whole: angle and time belong to one snapshot. Run TSan if supported.

• Test empty input, EOF with queued entries, a slow writer and write failure. Hold the writer with an explicit signal to fill the queue.

• Source silence, duplicates and NaN cause FAULT when the valid snapshot becomes too old. Test the 100/101 ms boundary with controlled clocks.

The watchdog measures time since receipt of a new snapshot at the host. It does not establish measurement freshness: data may have been delayed before reaching the reader. Clock synchronisation and source-delay estimation will be needed later.

Replay and processing latency

Repeat a failure and check whether monitor updates arrive on time

E1 Reprocess a motion recording

Save complete original scenario files and a separate cleaned log of accepted snapshots. Original data are needed to reproduce input errors. Write numbers without losing precision: in C++, use max_digits10 for double or preserve the original fields. Compare values and ordering, not CSV bytes.

Fast mode reads the file without waiting. Reanalysing raw data produces the same rejected and NO_RESPONSE events at the same seq values; analysing the cleaned file produces the same accepted states. Snapshots dropped by the queue are missing from the cleaned log but remain in the original scenario file. They cannot be recovered from the cleaned log alone.

E2 Check timing characteristics

Metric	Measurement boundary and meaning for the robot
Source interval	Difference between consecutive t_us values; 10 000 µs in the clean scenario. Gaps differ from host interval variation (jitter).
Age at monitoring	now_host-last_new_valid_receive_host. The watchdog uses this to detect channel silence.
Queueing and writing	t_done-t_enqueue on one monotonic clock. Shows logger queue accumulation.
Three-CLI runtime	From launch to completion on one file: includes runtime startup, parsing, reading and summary generation.
p50 p95 p99	Sort n latency values; take element ceil(p×n), counting from 1; p=0.50/0.95/0.99. Empty sample: null.

Clocks: Python perf_counter_ns, C++ steady_clock, Rust Instant. Subtract readings from the same clock: do not subtract t_us from host now. A 'nanosecond' unit does not imply that accuracy; a VM provides no hard deadline guarantee.

E3 Take measurements

• Generate one file of 100 000 motion snapshots; record SHA-256, seed and versions. First verify equivalence of the three CLIs. Use Release builds of C++/Rust without diagnostic tools.

• Use an external Python programme for 2 warm-ups and 10 measurements of each CLI, alternating languages. Save individual runtimes, median and range. Compare the implementations without claiming general language superiority.

• For C++, compare normal and slowed writers, K=4/64 and blocking/drop newest. Show maximum queue size, dropped, latencies and watchdog events. Collect at least 10 000 individual latency measurements for p99.

• In a 1-2 page report, explain whether history was retained, whether the monitor received updates, what limited processing, and whether the data allow the failure to be repeated.

Optional: replay 100-1000 snapshots at a specified rate: deadline_s=start_s+(t_us-first_t_us)/(1 000 000×speed), speed>0. Wait until each specified absolute time and record lateness. Control clocks in watchdog tests.

M1 first-half schedule

11-23 October • 20 hours per block • one test bench

Sunday has five 2-hour practice blocks. Weekdays cover reading, analysis and small checks; save results as experiment records. Dates and total workload follow the original plan with the clarification on page 1.

Date	Hrs	Task and deliverable
Sun 11.10	10	**Body and joint.** Block 1: environment, repository and L1. Block 2: axes, units, RobotState. Block 3: rest, tilt and motion generator. Block 4: Python parser and summary. Block 5: P3/P4 plots and manual verification. Deliverable: first experiment with plots.
Mon 12.10	2	**Linux and streaming.** 45 min paths and redirection; 45 min L2, pipe and EOF; 30 min shutdown checks. Deliverable: source can be stopped while preserving the log.
Tue 13.10	2	**Python and snapshots.** 45 min dataclass/exceptions; 45 min streaming reads; 30 min invalid fields and seq order. Deliverable: accepted-state update scheme.
Wed 14.10	2	**IMU data.** 45 min NumPy/axes; 45 min rest, roll and false tilt from added acceleration; 30 min plotting. Deliverable: understand accelerometer atan2 limitations.
Thu 15.10	2	**Experiment version.** 45 min branches and history; 45 min L3, axis-sign correction; 30 min L4 and lock. Deliverable: repeatable experiment with known parameters.
Fri 16.10	2	**Joint diagnostics.** 45 min P4 and the 9/10-sample boundary; 45 min P5, gaps and rejections; 30 min expectations. Deliverable: Python distinguishes the chosen scenarios.
Sat 17.10	0	Rest; no required tasks.
Sun 18.10	10	**C++ processor.** Block 1: compilation and C1. Block 2: C2 parser and summary. Block 3: C3 RAII logger. Block 4: C4 buffer and CMake. Block 5: comparison with Python. Deliverable: C++ validates and records snapshots.
Mon 19.10	2	**Snapshot ownership.** 60 min values, references and const; 40 min sensor buffer and consumer; 20 min conclusions. Deliverable: RobotState ownership diagram.
Tue 20.10	2	**RAII and logging.** 60 min lifetime, copy and move; 40 min write failure or exception; 20 min conclusions. Deliverable: explanation of file closure and error checks.
Wed 21.10	2	**STL and buffers.** 45 min vector/string_view; 45 min C4 and ASan; 30 min correction. Deliverable: snapshot survives input-row reuse.
Thu 22.10	2	**Building the test bench.** 60 min CMake targets; 40 min Debug/Release; 20 min commands. Deliverable: separate state library, CLI, logger and tests.
Fri 23.10	2	**Error investigation.** 45 min GDB; 45 min D1/D2, units and incorrect snapshot length; 30 min test. Deliverable: input, cause, correction and evidence.

By 23 October: both implementations accept and reject the same snapshots; the logger preserves values. The report shows axes, units and joint limits. Resolve discrepancies before porting the contract to Rust.

M1 second-half schedule

25 October - 3 November • 35 hours for faults, streaming and verification

Date	Hrs	Task and deliverable
Sat 24.10	0	Rest.
Sun 25.10	10	**Faults and Rust.** Block 1: scenarios/expected.json and S1-S5. Block 2: pytest/GoogleTest and T1/T2. Block 3: Cargo, RobotState and Result. Block 4: Rust validator. Block 5: shared verification programme and corrections. Deliverable: three CLIs check one contract.
Mon 26.10	2	**Evidence.** 45 min parametrisation; 45 min 9/10 exceedances, gaps and wrong units; 30 min T2. Deliverable: identify the test that catches each failure.
Tue 27.10	2	**Rust ownership.** 60 min ownership and borrowing; 40 min sensor row for analysis and logging; 20 min explanation. Deliverable: two ownership conflicts corrected.
Wed 28.10	2	**Invalid state.** 45 min Result/Option; 45 min NaN, out-of-range angle and Cargo tests; 30 min RAII/ownership. Deliverable: invalid snapshots do not update state.
Thu 29.10	2	**Communication between threads.** 60 min mutex/condition_variable; 40 min Q1 and closure; 20 min diagram. Deliverable: latest_state owners and overflow policy are defined.
Fri 30.10	2	**Update monitoring.** 45 min Q2 and clocks; 45 min T3, 100/101 ms boundaries; 30 min measurement plan. Deliverable: testable watchdog logic before thread integration.
Sat 31.10	0	Rest.
Sun 01.11	10	**Integration.** Block 1: reader/writer, queue and shutdown. Block 2: latest_state, monitor and FAULT. Block 3: Q3, overload and E1 replay. Block 4: E2/E3, raw measurements. Block 5: report and clean build. Deliverable: telemetry-core v1.0 candidate.
Mon 02.11	3	**Review and reserve.** 60 min conclusions on gaps, delays and causes; 60 min experiment instructions; 60 min one problematic scenario. Uses one additional programme hour.
Tue 03.11	2	**Test-bench verification.** 45 min explaining scenarios unaided; 45 min checklist; 30 min conclusions. Deliverable: skills and unmet criteria list.

Fitting integration into the budget

Before the final Sunday, prepare snapshots, a sequential logger, shared checks and a tested tick(now). Connect them with the watchdog in the main thread and reader/writer in worker threads. No separate task system or reactor is needed.

One joint, IMU axes, files and plots are required. Cut extensions first: noise, current plots, paced replay and additional input sizes. A complete robot, physics simulation, ROS, PID and an orientation filter come later. Record failed core tests as unfinished M1 work.

Final review: why can the log fall behind while the monitor receives new snapshots? Support your answer with counters and timestamps.

Assigned reading by task

Programming references and the compulsory first mechanism analysis

The plan and definitions are in English; references include English-language materials. Read the assigned passage before its task, then test the idea in code. You need not buy four books or read them cover to cover. Core reading is identified; consult the reference book when stuck.

Core reading

1. Scott Chacon, Ben Straub - Pro Git, 2nd ed. English edition. Sections 2.1-2.5: creating a repository, recording changes, history, undoing and remotes; 3.1-3.2: branching and merging. For L3/L4: axis-correction branch, threshold conflict and versioning a reproducible experiment.

2. Steve Klabnik, Carol Nichols and the Rust community - The Rust Programming Language. Rust Book. Chapters 4-6 and 9: ownership and references, structs, enum/match and errors for R1/R2. From 11-13: tests, CLI, file reading and iterators for R3/R4. Read chapters 3 and 8 selectively for syntax and collections; skim 16. Defer async, unsafe and FFI.

3. Anish Athalye, Jon Gjengset, Jose Javier Gonzalez - The Missing Semester of Your CS Education, MIT, 2020 course. Course notes. 'Course Overview + The Shell', 'Command-line Environment', 'Debugging and Profiling': redirection, processes, diagnostics and measurement for L1/L2, D1/D2, E3. Use relevant parts of 'Metaprogramming' for builds and dependencies.

C++ reference: specific sections

4. Bjarne Stroustrup - A Tour of C++, 3rd ed. Author and contents. §1.5-1.7: lifetime, const, pointers and references; §3.2 and 3.4: separate compilation and arguments; §6.2-6.3: copy/move and resources for C1-C3. §10.2-10.3 and 12.2: string, string_view and vector for C4. §18.2-18.4: threads, shared data and waiting for Q1-Q3. The book assumes programming experience; begin after P1/P2. Use the C++17 subset; modules and coroutines are unnecessary here.

Documentation for individual steps

Python Tutorial: Control Flow, Data Structures, Modules, Input/Output, Errors and Exceptions, Classes, Virtual Environments for P1/P2. Follow task-page links for NumPy/Matplotlib and testing tools. REP-103 (Units, Axis Orientation) and REP-145 (Data Sources) explain axes and accelerometers; reading them requires no ROS installation.

Motion fundamentals: what the programme describes

A **link** is a mechanism part treated as rigid in the model. A **joint** defines allowed relative link motion: a revolute joint permits an angle, a prismatic joint a displacement. A **degree of freedom** is one independent local motion. A fixed-base leg with an open link chain and three independent revolute joints has three; actuator count and degrees of freedom can differ.

Configuration q determines relative link placement; for a single fixed pivot it is an angle. A second-order mechanical model's state also includes velocity: x=(q, q_dot), where q_dot is angular velocity in rad/s. The same pose at different velocities gives different states. M1 RobotState is an observation snapshot, not the complete physical state.

Kinematics relates coordinates, velocities and accelerations without calculating forces. **Dynamics** explains changes in motion through mass, inertia, forces and torques. **Control** selects an input to achieve a goal. An inverse-kinematics angle does not establish that the motor can perform the motion. Further detail: M8, M9, M10.

Read before P1: Kevin Lynch, Frank Park, Modern Robotics, §2.1: configuration and degrees of freedom and §2.2: joints. Study definitions, joint types and the mechanism diagram; the general mobility formula comes in M5.

M1 key terms

Definitions for the data schema, resource ownership and stream processing

Telemetry. Transmission of measured quantities and diagnostic events to observe a system; it does not itself control motion.

State snapshot. A consistent set of values for one step; in M1, eight fields share a t_us timestamp.

Coordinate frame. An origin and oriented axes relative to which vector components are expressed; here x forwards, y left, z upwards.

Radian. A unit of plane angle: arc length divided by radius. One revolution is 2π rad; $\theta_{rad}=\theta_{deg}\cdot\pi/180$.

Specific force. Ideal accelerometer output: $f=a-g$; a is motion acceleration and g gravitational acceleration, expressed in the same frame. At rest with z upwards, $f_z\approx+9.81$ m/s².

Roll. Body rotation about the longitudinal x axis; the static estimate atan2(ay,az) is distorted by linear acceleration and cannot determine yaw.

Position error. Difference between requested and measured angles, $e=q_{ref}-q$, in radians; these quantities describe the command and observation respectively.

Timestamp. Event time on a chosen clock; source t_us and host time cannot be subtracted without aligning their timescales.

Monotonic clock. A timescale that never goes backwards; suitable for intervals within one time source, but it does not specify a calendar date.

Invariant. A condition that must hold in valid programme states; for example, accepted+rejected=rows_total after file processing.

Ownership. Responsibility for a resource's lifetime. The owner ensures release; a non-owning reference does not extend the resource's lifetime.

RAII. Tying a resource to a C++ object's lifetime: the destructor releases it. Object destruction does not prove a successful write.

Borrowing. Rust access through a reference without ownership transfer, limited by lifetime and compatibility rules for mutable and immutable access.

Dangling reference. A reference to an object whose lifetime has ended or whose address has become invalid; using it violates programme correctness.

Mutex. A mutual-exclusion mechanism that protects shared state from simultaneous conflicting access when all threads use it consistently.

Data race. Unsynchronised, conflicting thread accesses to the same memory, with at least one write; in C++, these cause undefined behaviour.

Backpressure. Slowing a producer when a consumer cannot keep up: in blocking mode, the reader waits for space in a full queue.

Watchdog. Periodic checking of the time since a new valid snapshot was received, using the host clock; silence causes FAULT without another message.

Latency. Time between two defined events, such as enqueue and write completion. Specify the boundaries and clock source before measuring.

p99 quantile. A value at or below which approximately 99% of observations lie; here, take element ceil(0.99·n) of the sorted sample, counting from 1.

Replay. Reprocessing saved input; raw rows are needed to reproduce rejections that are absent from the cleaned log.

SHA-256 hash. A 256-bit content fingerprint for checking file equality; it does not itself establish authorship or authentic provenance.

Check yourself: why is az≈g at rest; why does constant q not establish a fault's cause; why are dropped, rejected and seq gaps separate counters?

M1 equipment and setup

Prepare before L1, C1 and R1; setup time is included in the 75 hours

Equipment and purchases

Available: 1 MacBook Pro M4, a terminal and the existing 3D printer; choose its profile later using the exact printer model. M1 needs no printing. **Required purchases: none.** No IMU, MCU, motors, batteries, robot or GPU is needed: the Python generator supplies data. Borrow the C++ book from a library if available; purchase is optional.

Install before the first practical session: 11 October

On macOS ARM: an editor such as VS Code for Apple Silicon, and UTM. Use Ubuntu 24.04 ARM64 as the guest OS; choose the arm64 image and ARM virtualisation, rather than x86 emulation. Use an existing suitable Ubuntu installation if available. Allocate VM resources according to the Mac's memory and disk capacity; check free space and responsiveness before experiments.

In Ubuntu ARM64: Git, terminal and shell, ripgrep; uv and Python; numpy, matplotlib and pytest dependencies in one project. Pin Python and dependencies in pyproject.toml/uv.lock, and record commands in README. Do not install experiment packages in system Python. Save plots as PNG so experiments also run without a GUI.

Before C1: 18 October; before R1: 25 October

C++ in Ubuntu: GCC/G++ or Clang with C++17, CMake, make/Ninja, GDB, clang-format; a pinned GoogleTest version for tests. Ubuntu packages: build-essential, cmake, gdb, clang-format; record compiler choice and versions. Use a separate Debug configuration for ASan/UBSan; build TSan separately and only where the environment supports it.

Rust in the same Ubuntu: rustup, rustc, Cargo, rustfmt and Clippy. Pin the toolchain and Cargo.lock. Add serde/serde_json for R3 as needed. Native macOS builds are an additional check; take comparison measurements for all three CLIs in one Ubuntu environment.

Installation checks and optional additions

Record guest architecture (aarch64) and versions. Make a Git commit; use uv to import NumPy/Matplotlib/pytest and save a plot; build and run a C++17 example through CMake/CTest and stop at a GDB breakpoint; run cargo test, cargo fmt --check and cargo clippy. Restore dependencies from the lockfile in a clean copy and repeat L1/L4. The environment is ready for a task once its corresponding tool passes verification.

Optional: Docker only if isolation beyond the VM becomes necessary; M1 does not need it. Defer ROS 2, Gazebo and the GPU stack to their modules. Apple M4 does not support CUDA. A VM does not guarantee hard real time; check graphics and USB before later hardware experiments. Delivery time for subsequent purchases is outside study hours.

Basis: roadmap.pdf, physical p. 3 for the calendar; pp. 19-21 for M1 objectives. The RobotState schema, scenarios, teaching thresholds and acceptance checks were developed for this detailed plan. Installation and environment checks: p. 15.

First motion calculation: before generator P1

A planar link L=0.20 m pivots at the origin: x right, y upwards; q is measured anticlockwise from +x. Endpoint: p=(L cos q, L sin q). At q=0, obtain (0.20; 0) m; at q=0.50 rad, (0.17552; 0.09589) m. Mass is unnecessary for this calculation: it is forward kinematics.

In block 2 on 11 October, replace part of the RobotState analysis with 30 min for definitions and a diagram, and 30 min for calculation and plotting. Save mechanism.md and the figure; check ||p||=L and q=-0.50. Why can q and L alone not determine motor torque? Retain the CSV, [-1;1] rad limit and 75 hours.